Systems Security Assignment I

Mihai Bivol, Daniel Avram, Bence Soóki-Tóth

February 2026

Abstract

In this report we present the solution to Task 4 (Exploiting Elgamal Encryption) and 5 (Exploiting Encryption Oracles). We provide a detailed overview of our attacks, preliminaries and implementation.

Contents

1 Introduction **1**

2 Assignment (El Gamal) **2**

3 Preliminaries **2**
3.1 El Gamal encryption 2
3.2 Malleability of El Gamal encryption 2

4 Attack on malleable El Gamal implementation **2**
4.1 Implementation 3
4.1.1 Functionality 3
4.1.2 Usage 3

5 Assignment (RSA Encryption) **3**

6 Preliminaries **4**
6.1 RSA encryption 4
6.2 Malleability of RSA encryption 4

7 Exploitation of malleable RSA knowing the parity **5**
7.1 Implementation 5

1 Introduction

Benign implementations of cryptographic protocols can allow an adversary to break the security guarantees of any protocol. In this report we show how to break the textbook implementations of RSA with PKS 1.5 padding and El Gamal using properties such as residuity and homomorphism.

Notations. Table 1 provides the necessary syntax and notations used throughout the report.

Notation	Description
$\lvert x \rvert$	The length of x
$c_1 \mid\mid c_2$	The concatenation of two elements
$\mathsf{B}(\cdot)$	A function from $\mathbb{Z}$ to fixed number of bytes
$x\,[:l]$	The sub-bytes from 0 up to size l
$x\,[l:]$	The sub-bytes from size l up to $\lvert x \rvert$

Table 1: Notations used throughout this report.

2 Assignment (El Gamal)

The multiplicatively homomorphic properties of the textbook El Gamal acts as a double-edged sword, while providing ground to construct more efficient protocols [4, 3], an incautious implementation can be exploited by an adversary [2].

3 Preliminaries

3.1 El Gamal encryption

We recall the relevant syntax for El Gamal encryption [1], where we refer the security requirements to [1]. The scheme utilizes a group of prime order p, derived from the generator function Gen on input security parameter λ. It consists of three PPT algorithms:

Gen(1^λ): Pick a prime p *s.t.* $|p| = \lambda$, generator g, secret key $x \in [2, p-2]$, and compute public key $h = g^x \pmod p$.

Enc(m, h): Pick random $k \in [1, p-2]$, then compute ciphertext $(c_1, c_2) = (g^k \pmod p, m \cdot h^k \pmod p)$.

Dec(c_1, c_2): Given (c_1, c_2), recover message $m = c_2 \cdot (c_1^x)^{-1} \pmod p$.

3.2 Malleability of El Gamal encryption

The malleable property of textbook El Gamal usually refers to the homomorphic property of the scheme which allows an adversary $\mathcal{A}$ to exchange message m encrypted, to any message $m_{\mathcal{A}}$ of the adversaries choosing. The attacker proceeds as follows

1. $\mathcal{A}$ obtains ciphertext $c = (c_1, c_2) = (g^k, m \cdot h^k)$
2. $\mathcal{A}$ computes $a = m_{\mathcal{A}} \cdot m^{-1} \bmod p$
3. $\mathcal{A}$ set $c = (c_1, k \cdot c_2) = (g^k, m_{\mathcal{A}} \cdot h^k)$

thus completing the forgery.

4 Attack on malleable El Gamal implementation

The concrete implementation of the system attacked during the report provided and attack algorithm is presented in Figure 1.

$\mathsf{Enc}(m, x)$:

1. $k \xleftarrow{\$} \mathbb{Z}_p$
2. $c_1 = g^k,\ c_2 = m \cdot h^k$
3. **return** $c := \mathsf{B}(c_1) \mid\mid \mathsf{B}(c_2)$

$\mathsf{Forge}(m, m_{\mathcal{A}}, c)$:

1. $l = |c|$
2. $c_1 = \mathsf{B}^{-1}(c\,[: l]),\ c_2 = \mathsf{B}^{-1}(c\,[l :])$
3. $a = m_{\mathcal{A}} \cdot m^{-1}$
4. $c = c_1 \mid\mid a \cdot c_2$

Figure 1: El Gamal implementation attacked during our report, and the Forge function which allows and adversary to replace the message to any of their choosing.

4.1 Implementation

4.1.1 Functionality

The attack is implemented in Python and located at the following path in the repository au-syssec-f26-assignments/crypto/elgamal-encryption/forge.py containing the following main functions:

get_params(server: str). Returns the parameters available at `{server}/params`.

encrypt_via_endpoint(server: str, data_hex: str). Returns the encryption of `data_hex` using endpoint `{server}/encrypt_random_document_for_students/{data_hex}`

forge(orig_msg_hex: str, orig_ciphertext_hex: str, p: int). Returns a valid ciphertext with the original message `orig_msg_hex` replaced by any message using Forge from Figure 1.

4.1.2 Usage

The necessary packages can be installed using:

```
pip install -r requirements.txt
```

The attack script can be executed using the following command:

```
python3 forge.py https://elgamal.syssec.dk/
```

Which returns the cookie corresponding to the correct encryption of "You got a 12 because you are an excellent student! :)".

5 Assignment (RSA Encryption)

We are given the source code of a server with the following endpoints:

1. `'/'`: Returns a token that is the PKS 1.5 RSA encryption of a string

   ```
   f'You never figure out that "{secret}". :)'
   ```

   The value of `secret` is not known.

2. `'/quote'`: Receives a token and performs several checks on it:

(a) If `token == None`, redirects to `'/'`.

(b) If `token` is the encryption of an even number, returns

```
"I do not like even numbers."
```

(c) If the padding used when computing `token` was faulty, or the secret encrypted in `token` is not the right one, returns

```
"No quote for you!"
```

(d) If the token was computed with the right string:

```
f"{secret} because of weird oracles!"
```

returns a quote.

As we can see, the token returned by `'/'` endpoint was not computed from the right string. However, it still contains the `secret` value that could be used to compute the right token. Our task is to extract that `secret` value by exploiting the return values of `'/quote'` endpoint.

The server's implementation allows us to perform attacks based on either using a **Padding Oracle** or **Parity Oracle**. We decided to perform a **Parity Oracle attack**.

6 Preliminaries

6.1 RSA encryption

The RSA encryption relies on the assumption that it is difficult to factor large numbers. By computing two large primes, we can define a public-key cryptosystem that relies on this assumption:

Gen(1^λ) : Pick two primes p, q, both of $\lambda/2$-bit length. Set $n = pq$. Select a random number $e \leftarrow \mathbb{Z}^*_{(p-1)(q-1)}$ and set its inverse $d = e^{-1} \mod (p-1)(q-1)$. Output $pk = (n, e)$ and $sk = (n, d)$.

Enc(m) : $m^e \mod n$.

Dec (c) : $c^d \mod n$.

6.2 Malleability of RSA encryption

The malleability of this cryptosystem stems from the fact that the encryption function is a homomorphic one:

$$\mathrm{Enc}(m_1) \cdot \mathrm{Enc}(m_2) \equiv m_1^e \cdot m_2^e \equiv (m_1 \cdot m_2)^e \equiv \mathrm{Enc}(m_1 \cdot m_2) \mod n$$

By having the encryption c_1 without knowing the plaintext m_1, we can pick another plaintext m_2, compute its ciphertext c_2, and by multiplying the ciphertexts $c_1 \cdot c_2 \mod n$, we get the ciphertext for the plaintext $m_1 \cdot m_2$.

7 Exploitation of malleable RSA knowing the parity

Let's assume an adversary $\mathcal{A}$ that has access to a Parity Oracle lsb which, given the ciphertext, returns the least significant bit of the plaintext (in other words, it returns 0 if the plaintext is even, and 1 if it is odd). Thanks to the malleability of RSA encryption, $\mathcal{A}$ that intercepted a ciphertext $c = \text{Enc}(m)$ could multiply it by $Enc(2)$ to get the ciphertext for $2m$. Then, $\mathcal{A}$ can send this ciphertext to the oracle and get the parity of $2m$. This will reveal to the adversary some information about the interval that bounds the original plaintext m:

If $2m < n$, then the oracle will certainly output 0 (since we multiply by 2). This reveals that the original plaintext is bounded by $0 < m < n/2$.

If $2m > n$, then the oracle will certainly output 1, since n is odd, thus of the remainder of an even number will also be odd. This reveals that the original plaintext is bounded by $n/2 < m < n$.

$\mathcal{A}$ repeats this multiplication iteratively. At each iteration i, they shrink the interval that bounds m by half, depending on the output of $\text{lsb}(2^i m)$. This allows them to perform a binary search on the interval $(0, n)$ until they find the value of the plaintext m. Since they halve the search space at each iteration, the algorithm will run in $\mathbb{O}(\log_2 n)$ time.

The server that we are working with provides such an oracle by returning `"I do not like even numbers."` every time it receives a token that represents the encryption of an even value. Thus, we can perform the binary search to find the decryption of the token, unpad it, and extract the secret value:

Not using proper OAEP is dangerous ...

7.1 Implementation

The attack is implemented in Python and located at the following path in the repository `crypto/rsa-encryption/forge` containing the following main functions:

- `is_even(ciphertext)` sends the ciphertext to the server. If the response is `"I do not like even numbers."`, returns true. Returns false otherwise.
- `lsb_attack(ciphertext_hex, N, e)`: given the ciphertext and the public params, performs binary search using the `is_even()` function as a parity oracle. After finding the plaintext, it performs the unpadding opeartion, extract the secret value, and returns it.

References

[1] Taher ElGamal. A public key cryptosystem and a signature scheme based on discrete logarithms. In George Robert Blakley and David Chaum, editors, *Advances in Cryptology*, pages 10–18, Berlin, Heidelberg, 1985. Springer Berlin Heidelberg. 2

[2] Luca De Feo, Bertram Poettering, and Alessandro Sorniotti. On the (in)security of ElGamal in OpenPGP. Cryptology ePrint Archive, Paper 2021/923, 2021. 2

[3] Nicolas Huber, Ralf Kuesters, Julian Liedtke, and Daniel Rausch. ZK-SNARKs for ballot validity: A feasibility study. Cryptology ePrint Archive, Paper 2024/1902, 2024. 2

[4] Apurva K Vangujar, Buvana Ganesh, Alia Umrani, and Paolo Palmieri. A novel approach to e-voting with group identity based identification and homomorphic encryption. Cryptology ePrint Archive, Paper 2023/336, 2023. 2